

Documento Essenziale e Master Plan: Progetto "Switcha"

Corso di Tecnologie Web (TWeb) - A.A. 24/25 - Progetto Esteso (Versione Minima Essenziale)

Questo documento rappresenta la **guida operativa snella e senza ridondanze** per lo sviluppo di **Switcha**. Rispetta rigorosamente e al millimetro i vincoli fissati dal docente nel file `IndicazioniTweb.txt`, evitando qualsiasi espansione superflua del codice per mantenere la demo semplice, pulita e rapida da sviluppare e mostrare all'esame.

Stack Tecnologico Essenziale

- **Back-End:** Java con Spring Boot (`Spring Web`, `Spring Data JPA`, `H2 Database`, `Lombok`).
- **Front-End:** React + TypeScript con Vite, Vanilla CSS.
- **Ambito Benefit:** Tessitura di Comunità Civica ed Ecologia/Sostenibilità (baratto e scambio gratuito di favori/risorse senza denaro).

Matrice Esatta dei Requisiti (Senza Espansioni)

Componente	Requisito Fisso D'Esame	Conteggio Essenziale in <i>Switcha</i>
Documenti Teorici	3 Documenti + Slide	Visione (150-300 parole), Ideazione (2-3 pag), Sintesi (1-2 pag) + Slide PPTX
Entity Java	Esattamente 6 Entity	<code>User</code> , <code>Category</code> , <code>Announcement</code> , <code>Proposal</code> , <code>Message</code> , <code>Review</code>
Repository JPA	Esattamente 6 Repository	<code>UserRepository</code> , <code>CategoryRepository</code> , <code>AnnouncementRepository</code> ,

Componente	Requisito Fisso D'Esame	Conteggio Essenziale in <i>Switcha</i>
		<code>ProposalRepository</code> , <code>MessageRepository</code> , <code>ReviewRepository</code>
Service Spring	Esattamente 2 Service	<code>AnnouncementService</code> , <code>ProposalService</code> (esclusi login/user)
Controller Spring	Esattamente 2 RestController	<code>AnnouncementController</code> , <code>ProposalController</code> (esclusi login/user)
Route REST HTTP	Esattamente 12 Route	6 route in <code>AnnouncementController</code> + 6 route in <code>ProposalController</code>
Ruoli Utente	Esattamente 2 Ruoli	<code>ROLE_USER</code> (Utente Standard) e <code>ROLE_ADMIN</code> (Gestore)
Componenti React	Esattamente 8 Componenti	<code>Navbar</code> , <code>AnnouncementFeed</code> , <code>AnnouncementCard</code> , <code>AnnouncementDetailModal</code> , <code>CreateAnnouncementForm</code> , <code>ProposalModalForm</code> , <code>UserProfileView</code> , <code>AdminDashboard</code>
Form POST	Esattamente 2 Form	Form 1: Creazione Annuncio Form 2: Invio Proposta Scambio
Lifting State Up	Esattamente 1 Caso	<code>CreateAnnouncementForm</code> e <code>AnnouncementFeed</code> comunicano tramite il genitore comune <code>App.tsx</code>
Reattività Server-Client	Esattamente 1 Caso	L'accettazione di una proposta (PUT) aggiorna contemporaneamente la lista proposte e lo stato dell'annuncio visibile in bacheca



FASE 1: Documentazione Teorica

Step 1.1 — Documento di Visione (`Documento_Visione.txt`)

- **Lunghezza:** 150 - 300 parole.

- **Contenuto:** Descrizione di *Switcha* come piattaforma di scambio favori/competenze di quartiere a impatto sociale ed ecologico.

Step 1.2 — Documento di Ideazione (Documento_Ideazione.txt)

- **Lunghezza:** 2-3 pagine.
- **Contenuto:** 2 Scenari d'uso (Marco e Elena), 2 Attori (**USER** e **ADMIN**), Casi d'uso principali e selezione dei 2 casi da implementare (Pubblicazione annuncio & Invio proposta).

FASE 2: Setup e Data Layer (Backend Spring Boot - 6 Entity)

Step 2.1 — Struttura Cartelle

```
Tweb_Progetto/  
└─ Switcha/  
    └─ backend/      # Spring Boot (Porta 8080)  
    └─ frontend/     # React + TypeScript Vite (Porta 5173)
```

Step 2.2 — Le 6 Entity Java (@Entity)

Package: `com.switcha.backend.model`

1. **User:** `id`, `username`, `email`, `password`, `role` ("USER" / "ADMIN").
2. **Category:** `id`, `name`.
3. **Announcement:** `id`, `title`, `description`, `offeredService`, `requestedService`, `status` (ACTIVE, IN_TRATTATIVA, COMPLETED), @ManyToOne User, @ManyToOne Category.
4. **Proposal:** `id`, `status` (PENDING, ACCEPTED, REJECTED), @ManyToOne Announcement, @ManyToOne User applicant.
5. **Message:** `id`, `content`, `timestamp`, @ManyToOne Proposal, @ManyToOne User sender.

6. Review: `id`, `rating` (1-5), `comment`, `@ManyToOne` User `reviewer`, `@ManyToOne` User `reviewedUser`.

Step 2.3 — I 6 Repository JPA

Package: `com.switcha.backend.repository`

- `UserRepository`, `CategoryRepository`, `AnnouncementRepository`, `ProposalRepository`, `MessageRepository`, `ReviewRepository`.

Step 2.4 — Seed Dati Minimo (`data.sql`)

- 2 Utenti standard (`mario_user`, `elena_user`) e 1 Admin (`admin_boss`).
- 3 Categorie (*Giardinaggio*, *Informatica*, *Ripetizioni*).
- 3 Annunci di prova.

FASE 3: Logica e 12 Route REST (Backend Spring Boot)

Step 3.1 — I 2 Service Java (`@Service`)

Package: `com.switcha.backend.service`

1. `AnnouncementService`: Gestione ricerca, creazione, modifica e cancellazione annunci.
2. `ProposalService`: Gestione invio proposte, cambio stato e recensioni.

Step 3.2 — Le 12 Route REST Esatte (Suddivise su 2 Controller)

● Controller 1: `AnnouncementController` (`/api/announcements`) — 6 Route

1. `GET /api/announcements` — Lista annunci (QueryParam `?category=...`).
2. `GET /api/announcements/{id}` — Dettaglio annuncio (PathVariable).

3. **POST** `/api/announcements` — Creazione nuovo annuncio (RequestBody JSON) → **Form POST 1**.
4. **PUT** `/api/announcements/{id}` — Modifica annuncio (PathVariable + RequestBody).
5. **DELETE** `/api/announcements/{id}` — Cancellazione annuncio (PathVariable).
6. **GET** `/api/announcements/user/{userId}` — Annunci creati da uno specifico utente (PathVariable).

● **Controller 2: `ProposalController` (`/api/proposals`) — 6 Route**

7. **GET** `/api/proposals` — Lista proposte (QueryParam `?userId=...`).
8. **GET** `/api/proposals/{id}` — Dettaglio singola proposta (PathVariable).
9. **POST** `/api/proposals` — Invio nuova proposta (RequestBody JSON) → **Form POST 2**.
10. **PUT** `/api/proposals/{id}/status` — Accetta/Rifiuta proposta (PathVariable + QueryParam `?status=ACCEPTED`).
11. **POST** `/api/proposals/{id}/reviews` — Rilascio recensione (PathVariable + RequestBody JSON).
12. **DELETE** `/api/proposals/{id}` — Annullamento proposta (PathVariable).

Step 3.3 — CORS

Annotare entrambi i controller con `@CrossOrigin(origins = "http://localhost:5173")`.

FASE 4: Frontend React + TypeScript (Esattamente 8 Componenti)

Step 4.1 — Struttura degli 8 Componenti React

Cartella `src/components/`:

1. `Navbar.tsx` — Header con selettore del profilo utente corrente (Login semplificato `mario_user` / `admin_boss`).
2. `AnnouncementFeed.tsx` — Griglia principale contenente la bacheca annunci.

3. `AnnouncementCard.tsx` — Scheda sintetica del singolo annuncio.
4. `AnnouncementDetailModal.tsx` — Modale con i dettagli dell'annuncio selezionato.
5. `CreateAnnouncementForm.tsx` — **[Form POST 1]** Inserimento titolo, descrizione, cosa si offre e si cerca.
6. `ProposalModalForm.tsx` — **[Form POST 2]** Modale per inviare una proposta di scambio all'autore dell'annuncio.
7. `UserProfileView.tsx` — Gestione delle proposte inviate/ricevute dall'utente loggato.
8. `AdminDashboard.tsx` — Pannello essenziale riservato all'Admin per eliminare annunci.

Step 4.2 — I Requisiti Client-Side Chiave

- **Lifting State Up:** `CreateAnnouncementForm` e `AnnouncementFeed` condividono lo stato tramite il genitore `App.tsx`. All'invio del form, `App.tsx` aggiorna la lista e `AnnouncementFeed` mostra subito il nuovo annuncio.
- **Reattività Server-Client:** Cliccando "Accetta" su una proposta in `UserProfileView`, la chiamata `PUT` aggiorna sia lo stato della proposta che la lista degli annunci attivi in `AnnouncementFeed`.

FASE 5: Documento di Sintesi, Slide e Consegna

1. **Documento di Sintesi** (`Documento_Sintesi.txt`): 1-2 pagine su adeguatezza framework, deployment ipotetico (Render + Vercel), sicurezza e accessibilità.
2. **Presentazione Slide:** Compilare il file `Template_Progetto TWeb AA 24-25 - Slide.pptx`.
3. **ZIP Finale:** Inserire i 3 documenti `.txt` / `.pdf` e le slide nel file ZIP per Moodle. Codice pronto sul laptop per la discussione.